```
    return 0;
}

int f(int a)
{
    return a * 3.1416;
}
```

6. The **bool** data type stores Boolean values. The only values an object of type **bool** can have are **true** and **false**.

REVIEW SKILLS CHECK: Chapter 2

1.
```
#include <iostream>
#include <cstring>
using namespace std;

int main()
{
    char s[80];

    cout << "Enter a string: ";
    cin >> s;

    cout << "Length: " << strlen(s) << "\n";

    return 0;
}
```

2.
```
#include <iostream>
#include <cstring>
using namespace std;

class addr
{
    char name[40];
    char street[40];
    char city[30];
    char state[3];
    char zip[10];
public:
    void store(char *n, char *s, char *c, char *t, char *z);
    void display();
};

void addr::store(char *n, char *s, char *c, char *t, char *z)
{
    strcpy(name, n);
    strcpy(street, s);
    strcpy(city, c);
    strcpy(state, t);
```

```
    strcpy(zip, z);
}

void addr::display()
{
    cout << name << "\n";
    cout << street << "\n";
    cout << city << "\n";
    cout << state << "\n";
    cout << zip << "\n\n";
}

int main()
{
    addr a;

    a.store("C. B. Turkle", "11 Pinetree Lane", "Wausau", "In
       ", "46576");

    a.display();

    return 0;
}
```

3.
```
#include <iostream>
using namespace std;

int rotate(int i);
long rotate(long i);

int main()
{
    int a;
    long b;

    a = 0x8000;
    b = 8;

    cout << rotate(a);
    cout << "\n";
    cout << rotate(b);

    return 0;
}

int rotate(int i)
{
    int x;

    if(i & 0x8000)
        x = 1;
```

```
    else
        x = 0;

    i = i << 1;
    i += x;

    return i;
}

long rotate(long i)
{
    int x;

    if(i & 0x80000000)
        x = 1;

    i = i << 1;
    i += x;

    return 0;
}
```

4. The integer **i** is private to **myclass** and cannot be accessed inside **main()**.

2.1 EXERCISES

1.
```
#include <iostream>
using namespace std;

#define SIZE 100

class q_type
{
    int queue[SIZE]; // holds the queue
    int head, tail; // indices of head and tail
public:
    q_type(); // constructor
    void q(int num); // store
    int deq(); // retrieve
};

// Constructor
q_type::q_type()
{
    head = tail = 0;
}

// Put value on the queue.
void q_type::q(int num)
{
    if(tail+1==head || (tail+1==SIZE && !head))
```

```
    {
        cout << "Queue is full\n";
        return;
    }
    tail++;
    if(tail==SIZE)
        tail = 0; // cycle around
    queue[tail] = num;
}

// Remove a value from a queue.
int q_type::deq()
{
    if(head == tail)
    {
        cout << "Queue is empty\n";
        return 0; // or some other error indicator
    }
    head++;
    if(head==SIZE)
        head = 0; // cycle around
    return queue[head];
}

int main()
{
    q_type q1, q2;
    int i;

    for(i=1; i<=10; i++)
    {
        q1.q(i);
        q2.q(i*i);
    }

    for(i=1; i<=10; i++)
    {
        cout << "Dequeue 1: " << q1.deq() << "\n";
        cout << "Dequeue 2: " << q2.deq() << "\n";
    }

    return 0;
}
```

2.
```
// Stopwatch emulator
#include <iostream>
#include <ctime>
using namespace std;

class stopwatch
{
```

```
    double begin, end;
public:
    stopwatch();
    ~stopwatch();
    void start();
    void stop();
    void show();
};

stopwatch::stopwatch()
{
    begin = end = 0.0;
}

stopwatch::~stopwatch()
{
    cout << "Stopwatch object being destroyed...";
    show();
}

void stopwatch::start()
{
    begin = (double) clock() / CLOCKS_PER_SEC;
}

void stopwatch::stop()
{
    end = (double) clock() / CLOCKS_PER_SEC;
}

void stopwatch::show()
{
    cout << "Elapsed time: " << end - begin;
    cout << "\n";
}

int main()
{
    stopwatch watch;
    long i;

    watch.start();
    for(i=0; i<320000; i++); // time a for loop
    watch.stop();

    watch.show();

    return 0;
}
```

3. A constructor cannot have a return type.

2.2 EXERCISES

1.
```
// Dynamically allocated stack.
#include <iostream>
#include <cstdlib>
using namespace std;

// Declare a stack class for characters
class stack
{
    char *stck; // holds the stack
    int tos; // index of top of stack
    int size;  // size of stack
public:
    stack(int s); // constructor
    ~stack(); // destructor
    void push(char ch); // push character on stack
    char pop(); // pop character from stack
};

// Initialize the stack
stack::stack(int s)
{
    cout << "Constructing a stack\n";
    tos = 0;
    stck = (char *) malloc(s);
    if(!stck)
    {
        cout << "Allocation error\n";
        exit(1);
    }
    size = s;
}

stack::~stack()
{
    free(stck);
}

// Push a character.
void stack::push(char ch)
{
    if(tos==size)
    {
        cout << "Stack is full\n";
        return;
    }
    stck[tos] = ch;
    tos++;
}
```

```
// Pop a character.
char stack::pop()
{
    if(tos==0)
    {
        cout << "Stack is empty\n";
        return 0; // return null on empty stack
    }
    tos--;
    return stck[tos];
}

int main()
{
    // Create two stacks that are automatically initialized.
    stack s1(10), s2(10);
    int i;

    s1.push('a');
    s2.push('x');
    s1.push('b');
    s2.push('y');
    s1.push('c');
    s2.push('z');

    for(i=0; i<3; i++)
        cout << "Pop s1: " << s1.pop() << "\n";
    for(i=0; i<3; i++)
        cout << "Pop s2: " << s2.pop() << "\n";

    return 0;
}
```

2.
```
#include <iostream>
#include <ctime>
using namespace std;

class t_and_d
{
    time_t systime;
public:
    t_and_d(time_t t); // constructor
    void show();
};

t_and_d::t_and_d(time_t t)
{
    systime = t;
}

void t_and_d::show()
```

```
{
    cout << ctime(&systime);
}

int main()
{
    time_t x;

    x = time(NULL);

    t_and_d ob(x);

    ob.show();

    return 0;
}
```

3.
```
#include <iostream>
using namespace std;

class box
{
    double l, w, h;
    double volume;
public:
    box(double a, double b, double c);
    void vol();
};

box::box(double a, double b, double c)
{
    l = a;
    w = b;
    h = c;

    volume = l * w * h;
}

void box::vol()
{
    cout << "Volume is: " << volume << "\n";
}

int main()
{
    box x(2.2, 3.97, 8.09), y(1.0, 2.0, 3.0);

    x.vol();
    y.vol();

    return 0;
```

```
}
```

2.3 EXERCISE

1.
```
#include <iostream>
using namespace std;

class area_c1
{
public:
    double height;
    double width;
};

class rectangle : public area_c1
{
public:
    rectangle(double h, double w);
    double area();
};

class isosceles : public area_c1
{
public:
    isosceles(double h, double w);
    double area();
};

rectangle::rectangle(double h, double w)
{
    height = h;
    width = w;
}

isosceles::isosceles(double h, double w)
{
    height = h;
    width = w;
}

double rectangle::area()
{
    return width * height;
}

double isosceles::area()
{
    return 0.5 * width * height;
}

int main()
```

```
{
    rectangle b(10.0, 5.0);
    isosceles i(4.0, 6.0);

    cout << "Rectangle: " << b.area() << "\n";
    cout << "Triangle: " << i.area() << "\n";

    return 0;
}
```

2.5 EXERCISES

1.
```
// Stack class using a structure.
#include <iostream>
using namespace std;

#define SIZE 10

// Declare a stack class for characters using a structure.
struct stack
{
    stack(); // constructor
    void push(char ch); // push character on stack
    char pop(); // pop character from stack
private:
    char stck[SIZE]; // holds the stack
    int tos; // index of top of stack
};

// Initialize the stack
stack::stack()
{
    cout << "Constructing a stack\n";
    tos = 0;
}

// Push a character.
void stack::push(char ch)
{
    if(tos==SIZE)
    {
        cout << "Stack is full\n";
        return;
    }
    stck[tos] = ch;
    tos++;
}

// Pop a character.
char stack::pop()
{
```

```
    if(tos==0)
    {
        cout << "Stack is empty\n";
        return 0; // return null on empty stack
    }
    tos--;
    return stck[tos];
}

int main()
{
    // Create two stacks that are automatically initialized.
    stack s1, s2;
    int i;

    s1.push('a');
    s2.push('x');
    s1.push('b');
    s2.push('y');
    s1.push('c');
    s2.push('z');

    for(i=0; i<3; i++)
        cout << "Pop s1: " << s1.pop() << "\n";
    for(i=0; i<3; i++)
        cout << "Pop s2: " << s2.pop() << "\n";

    return 0;
}
```

2.
```
#include <iostream>
using namespace std;

union swapbytes
{
    unsigned char c[2];
    unsigned i;
    swapbytes(unsigned x);
    void swp();
};

swapbytes::swapbytes(unsigned x)
{
    i = x;
}

void swapbytes::swp()
{
    unsigned char temp;

    temp = c[0];
```

```
    c[0] = c[1];
    c[1] = temp;
}

int main()
{
    swapbytes ob(1);

    ob.swp();
    cout << ob.i;

    return 0;
}
```

3. An anonymous union is the syntactic mechanism that allows two variables to share the same memory space. The members of an anonymous union are accessed directly, without reference to an object. They are at the same scope level as the union itself.

2.6 EXERCISES

1. ```
   #include <iostream>
   using namespace std;

   // Overload abs() three ways:

   // abs() for ints
   inline int abs(int n)
   {
       cout << "In integer abs()\n";
       return n<0 ? -n : n;
   }

   // abs() for longs
   inline long abs(long n)
   {
       cout << "In long abs()\n";
       return n<0 ? -n : n;
   }

   // abs() for doubles
   inline double abs(double n)
   {
       cout << "In double abs()\n";
       return n<0 ? -n : n;
   }

   int main()
   {
       cout << "Absolute value of -10: " << abs(-10) << "\n";
       cout << "Absolute value of -10L: " << abs(-10L) << "\n";
   ```
   ```

```
    cout << "Absolute value of -10.01: " << abs(-10.01) << "\
       n";

    return 0;
}
```

2. The function might not be able to be in-lined because it contains a **for** loop. Some compilers will not in-line functions containing loops.

2.7 EXERCISES

1.
```
#include <iostream>
using namespace std;

#define SIZE 10

// Declare a stack class for characters
class stack
{
    char stck[SIZE]; // holds the stack
    int tos; // index of top of stack
public:
    stack()
    {
        tos = 0;
    }
    void push(char ch)
    {
        if(tos==SIZE)
        {
            cout << "Stack is full\n";
            return;
        }
        stck[tos] = ch;
        tos++;
    }
    char pop()
    {
        if(tos==0)
        {
            cout << "Stack is empty\n";
            return 0; // return null on empty stack
        }
        tos--;
        return stck[tos];
    }
};

int main()
{
    // Create two stacks that are automatically initialized.
```

```
    stack s1, s2;
    int i;

    s1.push('a');
    s2.push('x');
    s1.push('b');
    s2.push('y');
    s1.push('c');
    s2.push('z');

    for(i=0; i<3; i++)
        cout << "Pop s1: " << s1.pop() << "\n";
    for(i=0; i<3; i++)
        cout << "Pop s2: " << s2.pop() << "\n";

    return 0;
}
```

2.
```
#include <iostream>
#include <cstring>
#include <cstdlib>
using namespace std;

class strtype
{
    char *p;
    int len;
public:
    strtype(char *ptr)
    {
        len = strlen(ptr);
        p = (char *) malloc(len+1);
        if(!p)
        {
            cout << "Allocation error\n";
            exit(1);
        }
        strcpy(p, ptr);
    }
    ~strtype()
    {
        cout << "Freeing p\n";
        free(p);
    }
    void show()
    {
        cout << p << " - lengthj: " << len;
        cout << '\n';
    }
};
```

```
int main()
{
    strtype s1("This is a test."), s2("I like C++.");

    s1.show();
    s2.show();

    return 0;
}
```

MASTERY SKILLS CHECK: Chapter 2

1. A constructor is the function that is called when an object is created. A destructor is the function that is called when an object is destroyed.

2. ```
   #include <iostream>
   using namespace std;

   class line
   {
       int len;
   public:
       line(int l);
   };

   line::line(int l)
   {
       len = l;

       int i;

       for(i=0; i<len; i++)
           cout << '*';
   }

   int main()
   {
       line l(10);

       return 0;
   }
   ```

3. `10 1000000 -0.0009`

4. ```
   #include <iostream>
   using namespace std;

   class area_c1
   {
   public:
       double height;
   ```

```
    double width;
};

class rectangle : public area_c1
{
public:
    rectangle(double h, double w){ height = h; width = w; }
    double area() { return width * height; }
};

class isosceles : public area_c1
{
public:
    isosceles(double h, double w){ height = h; width = w; }
    double area(){ return 0.5 * width * height; }
};

class cylinder : public area_c1
{
public:
    cylinder(double h, double w){ height = h; width = w; }
    double area()
    {
        return (2 * 3.1416 * (width/2) * (width/2)) *
                (3.1416 * width * height);
    }
};

int main()
{
    rectangle b(10.0, 5.0);
    isosceles i(4.0, 6.0);
    cylinder c(3.0, 4.0);

    cout << "Rectangle: " << b.area() << "\n";
    cout << "Triangle: " << i.area() << "\n";
    cout << "Cylinder: " << c.area() << "\n";

    return 0;
}
```

5. An in-line function's code is expanded in line. This means that the function is not actually called. This avoids the overhead associated with the function call and return mechanism. Its advantage is that it increases the execution speed. Its disadvantage is that it can increase the size of the program.

6.
```
#include <iostream>
using namespace std;

class myclass
{
    int i, j;
```

```
public:
    myclass(int x, int y) { i = x; j = y; }
    void show() { cout << i << ' ' << j; }
};

int main()
{
    myclass count(2, 3);

    count.show();

    return 0;
}
```

7. In a class, members are private by default. In a structure, members are public by default.

8. Yes. It defines an anonymous union.

CUMULATIVE SKILLS CHECK: Chapter 2

1.
```
#include <iostream>
using namespace std;

class prompt
{
    int count;
public:
    prompt(char *s) { cout << s; cin >> count; }
    ~prompt();
};

prompt::~prompt()
{
    int i, j;

    for(i=0; i<count; i++)
    {
        cout << '\a';
        for(j=0; j<32000; j++) ; // delay
    }
}

int main()
{
    prompt ob("Enter a number: ");

    return 0;
}
```

2.
```
#include <iostream>
```

```
using namespace std;

class ftoi
{
    double feet;
    double inches;
public:
    ftoi(double f);
};

ftoi::ftoi(double f)
{
    feet = f;
    inches = feet * 12;
    cout << feet << " is " << inches << " inches.\n";
}

int main()
{
    ftoi a(12.0), b(99.0);

    return 0;
}
```

3.
```
#include <iostream>
#include <cstdlib>
using namespace std;

class dice
{
    int val;
public:
    void roll();
};

void dice::roll()
{
    val = (rand() % 6) + 1; // generate 1 through 6
    cout << val << "\n";
}

int main()
{
    dice one, two;

    one.roll();
    two.roll();
    one.roll();
    two.roll();
    one.roll();
    two.roll();
    return 0   ;
}
```